

PAPER 3 — HANDOFF v7.0

Written 2026-07-31. Supersedes v6.2 (2026-07-27).

Kourosh Jolaei — CoRo Lab, ÉTS Montréal. Supervisors: Vincent Duchaine, Jean-Philippe Roberge (JP).

Read this first. v7.0 covers 28–31 July 2026. Everything in v6.2 that is still true is repeated here, so this file alone is sufficient. Where v6.2 and v7.0 disagree, **v7.0 wins** — several v6.2 constants were deliberately changed.

>

The headline of this period: Block 2 is finished and validated, and a significant problem was found in the simulator that is not in my code — the tactile imprint does not encode object orientation the way geometry demands. That finding, not the pipeline work, is what needs a decision.

1. THE BIG PICTURE

1.1 The dissertation

Three papers on tactile-guided robotic grasping.

- **Paper 1** (JRM, published): shape-based contact extrapolation. A NN classifies the contact into a shape class; prototypical rules then extend the contact beyond the sensor edge.
- **Paper 2** (submitted June 2026 to *Robotics and Autonomous Systems*, Elsevier): "Tactile-Guided Regrasping: Virtual Search Optimization via Extrapolated Haptic Contact". CDT-based virtual regrasp search with GSR cost functions, 602 physical trials across 86 configurations.
- **Paper 3** (this work): **replace the hand-crafted classifier + extrapolation rules with a single learned contact-completion model**, trained on large-scale Isaac Sim data.

1.2 Why Paper 3 exists

Papers 1 and 2 both depend on a shape classifier: 94.3 % held-out, but only 80.8 % on external data. When it is wrong, every downstream step is wrong. JP's concrete example: a coffee container misclassified — 9 of 14 cylinder misclassifications. Paper 3 removes that failure mode entirely: one grasp in, extended contact map out, no shape class ever named.

1.3 The four blocks

Block	What	Status
1	Data collection in Isaac Sim (GUI + collector)	complete, validated
2	Stitching many grasps into an extended ground-truth map	complete, validated
3	Train the U-Net (sim-only, and sim+real)	not started
4	A/B against Paper 2's hand-crafted extrapolation	not started

1.4 Hardware / software stack

- Isaac Sim **5.1**, PhysX (GPU dynamics required — the pads are deformable bodies)
- UR5e + Robotiq 2F-85, two **TSF-85** tactile pads (Berith's extension)
- Each pad: **7 rows × 4 columns = 28 taxels**, 22 mm × 37 mm
- cuRobo for motion planning (injected via `sys.path`, **not** pip-installed)
- Tactile output: 28 values per physics frame at 60 Hz, via an ONNX CNN

2. THE OPEN PROBLEM — TILT IS NOT IN THE TACTILE MAP

This is the most important item in this document.

2.1 The observation

A flat sensor pad, held vertical, grasps a cylinder tilted about world X.

The contact line should lean by the same angle as the cylinder. It does not.

cylinder tilt	measured contact-line angle	fraction of geometry
0°	~0°	—
20°	~1°	5 %
35°	~16°	47 %

2.2 How it was measured

Two independent methods, agreeing:

1. **Row-centroid slope** (first attempt): per row, the pressure-weighted column centroid; least-squares line through the 7 points. Expected $\tan(20^\circ) \times \text{PITCH_Z} / \text{PITCH_Y} = 0.350$ col/row.

2. **Weighted-PCA blob axis** (`viz/blob_axis.py`, **preferred**): ported from Paper 2's own `virtual_search.generate_eigen_align`. Upsample $7 \times 4 \rightarrow 70 \times 40$ cubic, threshold at $0.35 \times 1.25 = 0.4375$ of the range, pressure-weighted covariance in **millimetres**, principal eigenvector. Validated against synthetic bands: 20° true $\rightarrow 19.81^\circ$ measured.

2.3 What has been ruled out

Hypothesis	Test	Result
Pad is not vertical	<code>pad_truth_probe.json</code>	orientation error 0.007°
Object moves during close	<code>object_moved_during_close_mm</code>	0.00 mm , every run
Cylinder too thin/thick	Ø20, Ø26, Ø50 all tested	no difference (~1° at 20° tilt on all)
Not pressing hard enough	4× indentation (<code>close_rad</code> 0.55 $\rightarrow$ 0.62)	angle +0.89° , unchanged; blob just got rounder (elongation 3.3 $\rightarrow$ 1.7)
Stitching loses it	measured per single grasp , before any stitching	absent at the source
Grid geometry / Y-offset	corrected sweep centring	no change
Object bolted vs kinematic	<code>GRASP_OBJECT_KINEMATIC=1</code>	NEVER SUCCESSFULLY RUN — still open

2.4 What is NOT ruled out

- **Kinematic object.** Berith specifically asked about this. Every attempt so far fell back to bolted (see §7.3). **This is the top loose end.**
- **Berith's CNN training distribution.** His paper (Frontiers, July 2025, §3.3.2) states the 13,000 training maps came from a ****fixed sensor** with a

Mark-10 dynamometer pressing 49 indenters straight down** — normal indentation only. All validation objects in Figures 1 and 7 are gripped **vertically**. Nothing published tests an obliquely-oriented contact.

Berith denies this is the cause (31 July call) but has not yet investigated.

- build v23c: rot+trans only, axis X, sign pred-act (both) — printed by the extension at startup. Meaning unknown; asked Berith, no answer yet. Could indicate the deformation is reduced along a fixed axis before the CNN.

2.5 The 20° vs 35° nonlinearity

5 % at 20° but 47 % at 35° is not simple attenuation. Two readings:

- **Real threshold** in the contact model — the interesting case.
- **Artifact**: at 35° the blob elongation drops to 1.24–1.43, near the ill-conditioned zone where the principal axis is unreliable. But note the four 35° measurements (two runs × two sensors) cluster at **+16.77, +14.65, +18.74, +15.45** — genuine outliers in this pipeline look like **+68°** and **−87.88°**, so this clustering argues it is real.

Unfilled gap: 25° and 45° have never been measured. Two single grasps, ~2 min each, would show whether the curve is smooth or has a step.

2.6 Berith call, 31 July 2026 — what he said

- Claims the CNN **was** trained for angled contact ("the one I put in there is just the most beautiful one"), contradicting my reading of his paper.
- Asked for my script + positions; offered to reproduce it himself.
- Suggested the object may need to be **kinematic** in Isaac.
- **Deformation files are not worth analysing** — "at one point just making garbage". Only the 28-value tactile output matters. (Drop that thread.)
- Everything current is for the **OLD sensor**, not the new one. Important for real-robot validation later.
- Migrating to **Newton** (MuJoCo-based) — force control, more stable kinematic chains — **ready end of August**. Too late for my schedule; will inherit later.
- Send via **Teams**.

3. VERIFIED CONSTANTS (carry these forward)

3.1 Geometry

PAD_W, PAD_H	= 22.0, 37.0 mm
N_ROWS, N_COLS	= 7, 4
PITCH_Y	= 5.5 mm (across the pad, 4 columns)
PITCH_Z	= 37/7 = 5.2857 mm (up the pad, 7 rows)
PAD_CENTER_ABOVE_CASE_M	= 0.0221

ROW 0 = the pad's PHYSICAL BOTTOM. Established 2026-07-22 by four independent judges; **independently confirmed 2026-07-30** by grasping only

the top of a rod (pad at $z = +80$, rod top at 1122.2, so only the pad's lowest 8.5 mm overlapped): rows r0+r1 carried **59 %** of the signal, r5+r6 carried **2 %**. All plots use `origin="lower"`.

3.2 Per-diameter calibration — Data/pad_offset_calibration.json

diameter	T00L_OFFSET_Z	close_rad	tactile_peak_sum
Ø20.0	0.15696	0.613	15177
Ø26.0	0.15651	0.55	13201
Ø50.0	0.1532	0.330	—

WARNING: Ø26 was overwritten during the hard-squeeze experiment with `close_rad 0.62 / T00L_OFFSET_Z 0.15654`. A backup exists at `Data/pad_offset_calibration_BACKUP.json`. ****Verify Ø26 reads 0.55 before trusting any new Ø26 run.****

`close_rad` estimate for a new diameter: $\text{rad} \approx (85 - D_{\text{mm}}) / 106$

(jaw span is near-linear in the joint angle; reproduces Ø26 $\rightarrow$ 0.557 vs true 0.550).

3.3 Processing conventions (CHANGED from v6.2 — read carefully)

```

HOLD_FRAC          = 0.9          # was 0.5 in v6.2
SUBTRACT_BASELINE = True          # was False in v6.2
MIRROR_S2_IN_OVERLAY = False
OUTLIER_MM         = 8.0
INITIAL_GRASP      = "first"      # new: "first" | "center" | "ptNN"
res_mm             = 0.75         # standard stitch resolution

```

- **HOLD_FRAC 0.5 $\rightarrow$ 0.9.** At 0.5 the averaging window reached down to

62.6 % of peak, i.e. into the closing ramp. Paper 2 averaged a fixed 1 s

window entirely inside the steady grasp (`tactile_DataReadSave3.run_average`,

~200 samples at 200 Hz, plain `np.mean`). 0.9 gives 91.8–100 % of peak,

210 of 216 frames — same convention, no ramp contamination.

- **SUBTRACT_BASELINE False $\rightarrow$ True.** Removes ~1.3 % of a map and, more

importantly, the unphysical negatives (map min went from **-32.7 $\rightarrow$ 0.0**)

that sit exactly at the fade edges where the silhouette boundary lives.

Baseline is measured **per grasp** from the pre-contact frames.

Does NOT fix the row-gain spread (46 % $\rightarrow$ 48 %) — an earlier claim of

mine that the data disproved.

3.4 Baseline is constant (measured, 21 grasps)

```

per-grasp baseline sum: mean 267.0 std 1.5 (0.6 %)
per-taxel std across grasps: 0.14 a.u. (~0.01 % of a 1070 peak)

```

So per-grasp and Paper 2's fixed 30-second vector are **equivalent here**.

Per-grasp is the more robust choice for real-robot work (no stale calibration

file). This is a footnote, not a limitation.

4. VINCENT & JP MEETING, 28 JULY 2026 — decisions that shape Block 3

4.1 The training pair must be re-shaped

Vincent was explicit: the ****input** must sit inside the extended frame,

symmetrically, **extended by about** two taxels each way** (his example:
 $7 \times 4 \rightarrow 11 \times 8$). In mm: $11 \times 5.286 = \mathbf{58.15 \text{ mm}}$ (Z) $\times 8 \times 5.5 = \mathbf{44.0 \text{ mm}}$ (Y),
so the sweep must carry the pad centre **$\pm 10.57 \text{ mm}$ in Z and $\pm 11.0 \text{ mm}$ in Y.**

Key geometric fact: extension per side = **half** the grid travel
(the pad already covers $\pm 11.0 / \pm 18.5 \text{ mm}$). So 2 taxels all round needs
22.0 mm of Y travel and 21.1 mm of Z travel, on an **odd $\times$ odd** grid.

4.2 Only centre frames as inputs

I proposed reusing all N grid positions as separate inputs against the same stitch. **Vincent rejected it:** off-centre inputs add a positional variable ("tell me where you are in that frame and extrapolate in a way that fits"), the extension stops being symmetric, and it needs more data, not less.

Not yet raised with him: a **sliding window** over a larger grid would recover most of the lost yield without violating his objection — on a 7×7 grid each interior 3×3 point has full ± 2 -taxel coverage *around itself*, so every pair still has a genuinely central input. Cost drops from 25 grasps/pair (5×5) to 5.4 (7×7) to 3.2 (9×9). **Worth proposing.**

4.3 JP's layer problem, and Vincent's resolution

JP: tactile stitching is not image stitching — you only get the contact surface, no depth. As the pad passes the object's maximum, contact appears that has no counterpart in the earlier frames (his plus-shape example).
Vincent's resolution: stay in 2D, and stop the sweep whenever the data forces a plane change, detectable because the gripper closes further.

This requires contact-aware closure (§9.2), because the collector currently closes blind to a fixed `close_rad` — gripper opening carries zero information today.

4.4 Throughput is now a named risk

2–3 min/grasp on the laptop $\rightarrow$ Vincent's own arithmetic: 720 grasps/day, and that was for a 1-D sweep. Options: headless, Newton, Calcul Québec. His caution: **don't fork Berith's Newton work mid-flight.**

4.5 LLM benchmark — done informally

Vincent tried Gemini and GPT-5.6 on tactile extrapolation: ****both failed outright****. Claude did best — but by treating the map as a numeric matrix and extrapolating gradients, with **no sensor knowledge and no shape priors**.
Vincent's read: that is an argument *for* a learned model, not against.

4.6 Other

- Vincent independently corroborated the hinge explanation for uneven pressure, and added that real grippers **flip** which end presses harder as they wear (~100k cycles). Good limitations-section material.

- Meetings Tuesdays and Fridays. **Next: 11 a.m., 4 August** (Vincent in Europe).

5. WHAT WAS BUILT AND FIXED THIS PERIOD

5.1 viz/stitching.py

1. **4th column** — `_composite_extended()`. Initial contact heat-mapped in the middle, stitched extension around it, in one array. This is the figure Vincent asked for. Reuses `res["paint"]` so it lands on the same grid as the canvas and cannot drift. Reports extension **per side** (up/down/left/right), never assuming symmetry.

2. **INITIAL_GRASP selector** ("first" / "center" / "ptNN").

On a centered grid `pt00` is the centre, so both agree.

3. **Exact tiling** (`_splat_one` **rewritten**). Was: fixed `round(PITCH/res)` blocks — at 0.75 mm/cell that is `round(5.5/0.75) = 7` cells = **5.25 mm** against a 5.5 mm pitch, leaving an **unpainted gap column** and a pad **21.0 mm wide instead of 22.0**. Now: each canvas cell takes the value of the **nearest taxel centre** (Voronoi on the taxel lattice), masked to the pad footprint. ****Verified: 100 % of the footprint painted, 0 unpainted cells****, taxel widths 8/7/8/7 cells summing to exactly 30.

> ****Consequence: overlap sigma before and after this change are not comparable.**** Sigma from 2026-07-29 onward is the trustworthy one.

4. `stitch_report.txt` — a `_Tee` mirrors stdout to `<run>/Stitched/stitch_report.txt` with a header recording every constant in force. `stitch_run()` returns **PNGs only** (the GUI `imread()`s the return value, so a `.txt` in that list would crash the button).

5. `_read_tactile_csv()` — tolerates the CSV writer race (§6.3), skips bad lines, reports the count, warns loudly above 5 %.

6. `save_hold_averages()` → `<run>/hold_average_maps.csv`, one row per grasp per sensor: ``grasp, sensor, n_hold_frames, peak_sum, map_sum, hold_frac, baseline_subtracted, t_r0c0 ... t_r6c3`` (row-major, `r0` = bottom). Written by **both** the Stitch and Heatmaps buttons.

7. `HOLD_FRAC` → 0.9, `SUBTRACT_BASELINE` → True (§3.3).

5.2 main_gui.py

1. **Centered grid checkbox** — `n` becomes steps **per side**, so an axis holds $2n+1$ points and the initial pose sits in the middle.

$(2 \cdot nx+1) \times (2 \cdot ny+1)$ — **$nx=ny=2$ gives 25 points, not 12.**

2. **Serpentine (boustrophedon) order** — see §6.2. Live label shows point count, estimated runtime and **max jump**, red above 15 mm.

3. **Object diameter and length fields** with a live calibration status line (green = calibrated with its constants; red = not calibrated, with the

close_rad estimate). Replaces the hard-coded CYL_D = 26.0.

4. close_rad **field** + **"Use estimate for this diameter"** in the Calibrate tab; baked into the generated command as GRASP_CLOSE_RAD="...".

5. **Scrollable left panel** (the Collection tab had grown past the window).

6. Finer FRONT-preview ticks: 10 mm major / 2 mm minor, tunable at the top of the file.

5.3 sim/collect_from_config.py

1. **Object size from the config.** Object_02/Cylinder is a **UNIT mesh** — its real size is entirely the transform (scale = (D, D, L), translate.z = L/2). So a new diameter is a number, **no new USD, no STL**. Prints [obj] size set from config: D=... -> scale (...).

2. CLOSE_RAD **read from the calibration store** per diameter. The store was already *writing* close_rad; it was simply never read back. GRASP_CLOSE_RAD=<rad> overrides (needed for the first grasp on a new diameter).

3. **Calibration guards.** Refuses to store when the tactile peak is outside 0.5×–2.0× the Ø26 reference (band **6600–26400**) or when the object moved, printing the corrected close_rad and the change in mm. Exit codes: **5** = peak out of band, **6** = object moved.

GRASP_CAL_FORCE=1 overrides.

4. GRASP_OBJECT_KINEMATIC=1 switch (§7.3 — not yet working).

5. OBJECT_POSE_CHECK after world.reset(), on every run: reads the rod's axis back and reports its angle from world Z vs what was asked.

5.4 viz/validation.py

- _sample_canvas **rewritten** to be the exact inverse of the new splat.

Verified: max recovery error ~1e-13, SSIM 1.000000 at 1.0 / 0.75 / 0.5 / 0.25 mm per cell.

- overlap(x) **column** — mean grasps painting each grasp's own footprint.

Makes SSIM interpretable: ~1.0 is expected **only** at overlap ~1.0.

- **GSR saturation warning.** On rigid-cylinder runs GSR reads ****100.00 %** for every grasp**, so GSR_err = 0.00 by construction — zero dynamic range, no information. It now says so loudly. ****Do not quote GSR as validation on this data.****

5.5 New tools

File	What it does
viz/blob_axis.py	Paper-2 weighted-PCA blob axis; per-grasp angle, elongation, cell count, plus mean angle (real tilt) and angle vs lateral Y (position artifact). **Glob is _pt_s1_tactile_maps.csv** — files without a _pt tag are silently skipped.
viz/repeat_compare.py	Two runs of identical config compared at three levels: per-grasp 7×4 maps, pad placement, and the stitched canvas aligned on pt00.

File	What it does
viz/flatfield.py	Row-gain from a straight-cylinder run + measure_tilt_slope (per-Z-row Y centroids, fitted line, slope/angle/R ²).

5.6 viz/heatmaps.py, viz/temporal_snapshots.py

- heatmaps.py now **delegates to** stitching.hold_average. It previously kept its own copy with a **different hold threshold** ($0.5 \times \text{peak}$ vs $\text{min} + 0.5 \times \text{range}$) and no baseline handling — so the heatmap you looked at was not guaranteed to be the map that got stitched. Verified identical now.
- temporal_snapshots.py: one **shared colour scale** across all grasps and stages (Vincent's 21 July objection), and baseline subtraction applied **before** finding the 5/50/95 % crossings.

6. MEASURED RESULTS

6.1 Repeatability — 2.4 % RMS

Two runs, identical config (15-point centered grid, Ø26, tilt 0):

level	s1	s2		
per-grasp 7×4 maps, mean RMS	2.51 % of peak	2.13 %		
pad placement, mean \	Δpos		0.303 mm	0.303 mm
stitched canvas, RMS	2.36 %	2.03 %		
overlap sigma	123 → 122	118 → 114		

Level 1 ≈ Level 3 means stitching adds essentially no noise. Mean drift

$dy = -0.002$, $dz = +0.001$ mm — pure scatter, no bias.

The number to quote: a training pair is reproducible to ~2.4 % RMS.

That is the noise floor of the ground truth; no model can be asked to beat it.

6.2 The 22.7 mm jump bug → serpentine fix

Placement was **binary, not jittery**: 13 of 15 grasps agreed to **< 0.05 mm**,

two were off by **2.2 mm** — and in each run a *different* point, both by the identical vector (−0.54 mm Y, +2.17 mm Z). Both followed the only two

22.7 mm pad-to-pad moves in the run: the raster column wrap

($\text{hypot}(22.0, 5.5)$).

Fix: reverse every other Y-column so the sweep snakes. Max jump

22.7 → 12.3 mm; path 116 → 89 mm. Stitching is order-independent (a sum),

so nothing downstream changes.

Not yet verified — no repeatability run has been done since the fix.

6.3 CSV writer race (Berith's extension)

Roughly **one bad line per run**. Two writes collide into one line:

```
101.933339,6116102.816672,6169,109.93...
```

Row A began 101.933339, 6116 and stopped mid-field; row B was appended onto it, fusing frame 6116 with time 102.816672 → 31 fields instead of 30.

Corrupts **both** the per-grasp and the cumulative file from one event.

Seen in s1 both times (2 for 2) — worth telling Berith.

All four post-processing modules now skip and report instead of crashing.

6.4 Taxel row gain — 59–60 % spread, and it is NOT the hinge

Row profile from a straight-cylinder run (row 0 = bottom):

s1:	308	482	305	500	563	482	403	spread 59 % of mean
s2:	287	441	285	505	532	448	391	spread 60 %

On a straight vertical rod every row presses on identical geometry, so this is **instrument, not object**. Brightest/dimmest = **1.85×**.

It zigzags — dips at rows 0 *and* 2 with bright rows either side, peak at row 4 not at the top. A hinge is a lever and can only produce a smooth ramp.

So the earlier "hinged finger" explanation is wrong for the shape; this looks like **per-taxel gain variation in Berith's ONNX model**.

- `row_gain.json` is saved by `flatfield.py` and is the correction.
- **Correcting it does not change the measured tilt** ($0.134 \rightarrow 0.128$) — a multiplicative row gain cancels out of a per-row centroid. The correction is **cosmetic**, useful for figures only.
- Caveat: measured at one grip closure on one Ø26 rod. If taxel response varies with pressure, the correction is only valid near that operating point.
- **A 7-vector only fixes rows**. If individual *taxels* vary, the pattern is 7×4 — which a flat plate would measure (§9.1).

6.5 Contact geometry — the compliance-limited silhouette

On Ø26, contact spans only about **±7.5 mm** and then dies, against a geometric edge at ±13 mm. Half-width $\approx \sqrt{(2R\delta)}$ gives $\delta \approx$ **1.16 mm** of indentation. Covering the full 22 mm pad would need $\delta \approx$ 4.65 mm.

So what you image is the **compliance-limited contact boundary**, not the 26 mm outline. That fade-to-zero edge is real, learnable structure — it is the first genuine boundary to appear inside the extension region.

6.6 Diameter does not change blob elongation (20 vs 26 mm)

Mean elongation: Ø26 = 3.38 / 3.43, Ø20 = 3.30 / 2.62. No meaningful difference, slightly *higher* for the thicker rod. This tested the hypothesis that a thinner rod gives a more directional imprint — **null result** over that range. Ø50 has been calibrated but the tilt series was never run.

Note on discarding: grasps with **elongation < 1.5** give meaningless angles (observed: +68.21°, -87.88°, +86.84°, all at elongation ~1.1). `blob_axis.py` reports elongation but **does not yet refuse** to report an angle below 1.5 — that guard is still to be added, and without it those outliers poison a mean.

7. THINGS TRIED THAT DID NOT WORK

7.1 Berith reproducer script — 5 failed attempts

Goal: a short self-contained script Berith could run. Each attempt failed differently:

1. Bare `import TSF_85_Ext` — extensions load through the `**extension manager**`, not a plain import. No CSVs at all.
2. Missing GPU settings → Deformable Body feature is only supported on GPU; pads never simulated. Fixed by setting `/physics/enableDeformableBodies` and `/physics/enableGpuDynamics` **before** any Isaac import, plus `enable_gpu_dynamics(True)` / `set_broadphase_type("GPU")` on the context and `PhysxSceneAPI` on the prim.
3. Teleporting the arm to `Q_GRASP` without cuRobo's **FINAL TRIM** → pad lands mm off, one pad grazes and the other digs in (peaks 10.4k vs 15.4k; tilt-0 gave only **2 hold frames**).
4. 600 settle frames fixed the arm (joint error 0.0088° , pads level to 0.107 mm) but the **fingers never closed** — pad X gap stayed at 125 mm (open) instead of ~67 mm.
5. Switching to two indexed `apply_action` calls (matching the pipeline) **still** did not close the fingers.

Unresolved and worth knowing: the pad Case readback returned *identical* values before and after the close. That may mean the fingers genuinely did not move — **or** that `XformCache` reads authored USD that PhysX never writes back without fabric sync, making the diagnostic itself unreliable. **This was never settled.** The deciding test is one command: check `hold_average` peak on that run — ~13,000 means the fingers did close and only the diagnostic was wrong; ~270 means they did not.

Decision taken: stop, and send Berith

`viz/kourosh_pipeline_summary.py` — a ~140-line **readable summary** with all exact numbers, explicitly labelled as documentation not a runnable collector — plus the real tactile CSVs, heatmaps and GUI screenshots.

7.2 Wrong Y-offset advice (my error, corrected)

For a centered grid the pad offset **is** the sweep centre, so on a tilted rod it must be $y = -z \cdot \tan(\text{tilt})$ — at $z=30$, tilt 20° : **-10.9**, not -16.4.

`run_20260729_182002` and `run_20260730_160150` swept **off-centre** as a result (166.1 → 199.1 against a rod centre at 199.0). Per-grasp blob results are unaffected; the **stitched maps from those runs are offset**.

7.3 Kinematic object — never successfully run

Three attempts:

1. Flag set, but the rod **snapped upright** — the tilt was being applied by the **fixed joint's** `LocalRot0`, not by the prim transform, so removing

the joint removed the tilt.

2. Authored the transform explicitly → AddXformOp **precision error**:

existing xformOp:orient is quatd, requested PrecisionFloat.

Fell back to bolted.

3. Stripped back to flag-only + a post-world.reset() readback. ****Not yet run.****

Still the top loose end, and the thing Berith asked about.

7.4 Environment gotcha — the JIT hang

cuRobo compiles five CUDA extensions on first run. Root causes found:

- **stale lock file** in ~/.cache/torch_extensions/py311_cu128/geom_cu/ — torch's FileBaton waits forever for a lock nobody will release. Survives reboot. Delete it.
- TORCH_CUDA_ARCH_LIST **unset** → builds for every arch. GPU is **sm_75**; export TORCH_CUDA_ARCH_LIST="7.5" (now in ~/.bashrc).
- **Parallel build overwhelms the machine** → export MAX_JOBS=2.
- Full reset if it recurs: rm -rf ~/.cache/torch_extensions/py311_cu128, then let it rebuild ~15 min untouched.
- Do **not** source ~/isaacsim/setup_python_env.sh — it points PYTHONPATH at Isaac's Python 3.11 and breaks the system pip in that shell.
- pxx is only importable **after** SimulationApp starts; for standalone USD inspection use pip install usd-core.

8. PAPER 2 CONVENTIONS — sim/real comparability (matters for Block 4)

Located at /home/kourosh/Pipeline_ws/ros2_ws/Python_Modules/.

step	Paper 2	Paper 3 sim	match?
frame selection	run_average(duration=1): ~200 samples at 200 Hz, plain np.mean, taken during the steady grasp	210 frames ≥ 90 % of peak	yes (after the HOLD_FRAC change)
baseline	fixed vector = mean of Free_30_Seconds_Data.csv	per-grasp, pre-contact frames	equivalent (§3.4)
spatial smoothing	Gaussian $\sigma_x = \sigma_y = 1.2$ on each 7×4 block	none	NO
normalisation	clip at 3000, divide by 3000 (norm_block)	raw a.u. (0–1200 range)	NO
blob axis	weighted PCA in mm after 10× cubic upsample	same (blob_axis.py)	yes

Open decision for Vincent: should Paper 3's model inherit Paper 2's full preprocessing chain, or deliberately depart from it and document why?
 $\sigma=1.2$ on a 4-wide array is heavy blur. If Block 3 trains on unsmoothed sim maps and Block 4 compares against Paper-2-processed real maps, that is an apples-to-oranges problem baked in from the start.

Note: the GSR path in validation.py is already correct — it hands raw

56 values to `predict_grasp_success`, which applies all three steps internally.

9. PARKED WORK

9.1 Flat-plate flat-field correction

Grasp a **26 mm-thick flat plate** — the one object where the fixed `CLOSE_RAD` still applies, since you choose the thickness. Every taxel should read the same; the resulting 7×4 is the full per-taxel correction map (better than the 7-vector, §6.4). Deferred to the object-replacement work.

9.2 Contact-aware closure (was §9.4 in v6.2)

Currently the collector closes blind to a fixed `close_rad`. Contact-aware closure would watch the tactile sum **during** closing and stop at a target, making `close_rad` an **output** rather than an input.

Needed for: JP's plane-change stop rule (§4.3), arbitrary objects without per-diameter calibration, and the flat plate.

Not needed for: single controlled diameters — the per-diameter calibration store already handles those. **This is no longer blocking §9.1 or new objects.**

Caveat for when it is built: "equal peak" is not "equal grasp" — on a soft or thin object the same peak means much deeper indentation.

9.3 Adding new objects — SOLVED, no STL needed

`Object_02/Cylinder` is a **unit mesh**; size is entirely the transform. The

GUI diameter/length fields drive it end to end. A new diameter is:

set diameter → Calibrate tab → "Use estimate" → run → status turns green.

`Object_01` and `Object_03` also exist in the scene with their own Looks — **never inspected**. They may already be a small object library.

10. WHERE THINGS ARE ON DISK

<code>~/Paper3_Simulation/</code>	
<code>main_gui.py</code>	GUI (Collection / Calibrate / Stitching tabs)
<code>sim/collect_from_config.py</code>	the collector (~1700 lines)
<code>viz/stitching.py</code>	canvas, composite, hold_average, tolerant CSV
<code>viz/validation.py</code>	round-trip SSIM / TC / GSR
<code>viz/heatmaps.py</code>	per-grasp figures (delegates to stitching)
<code>viz/temporal_snapshots.py</code>	4-stage squeeze snapshots
<code>viz/blob_axis.py</code>	NEW — Paper-2 weighted-PCA blob axis
<code>viz/repeat_compare.py</code>	NEW — run-to-run repeatability
<code>viz/flatfield.py</code>	NEW — row gain + tilt slope
<code>viz/kourosh_pipeline_summary.py</code>	NEW — the readable summary for Berith
<code>Data/pad_offset_calibration.json</code>	per-diameter calibration
<code>Data/pad_offset_calibration_BACKUP.json</code>	pre-hard-squeeze backup
<code>Data/gui_run/run_YYYYMMDD_HHMMSS/</code>	one folder per run
<code>objects_stl/, objects_usd/, scenes/</code>	(STL route not needed)
<code>TSF-85/</code>	Berith's extension (submodule)
<code>curobo-stable/</code>	cuRobo (submodule)

Per-run outputs: `gui_ptNN_s{1,2}_tactile_maps.csv` (~325 rows),

`pose_history.json`, `pad_truth_probe.json`, `reachability_report.json`,

`run_progress.log`, `nan_watch.tsv`, `gui_config_used.json`,

`hold_average_maps.csv`, `Stitched/`, `Heatmaps/`, `Temporal_Per_Grasp/`.

Key runs

run	what
run_20260723_183658	first verified training pair
run_20260729_132515	15-pt centered, Ø26, tilt 0 — repeat A
run_20260729_143021	same config — repeat B (2.4 % RMS pair)
run_20260729_210524	21-pt centered, tilt 0 — best sigma 92/95 ; the straight-rod reference
run_20260729_182002	21-pt, tilt 20°, Ø26 (off-centre Y)
run_20260730_160150	21-pt, tilt 20°, Ø20 (off-centre Y)
run_20260730_190107	single grasp, tilt 20°, 4× indentation
run_20260731_130722	single grasp, tilt 35° , bolted — angle +16.8/+14.7
run_20260731_135421	single grasp, tilt 35° , repeat — angle +18.7/+15.5

Git: github.com/KouroshJolaei/Paper3_Simulation, branch main.

Data/ is gitignored (**runs are not backed up by pushing**). `__pycache__`

untracked as of this period. TSF-85 and curobo-stable are **submodules** —

`git add .` from the root does **not** capture edits inside them.

11. NEXT STEPS, IN ORDER

Immediate (before or at the 4 August meeting)

1. **Send Berith the package** (Teams): `kourosh_pipeline_summary.py`, the straight and 35° tilted `gui_pt00_s{1,2}_tactile_maps.csv`, both heatmap PNGs, the GUI screenshot. One line: *Ø26 rod, pad vertical to 0.007°, object movement 0.00 mm — blob axis ~1° at 20° tilt, ~16° at 35°, reproduced across two runs and on Ø20/Ø26/Ø50.*

2. **Restore the Ø26 calibration** from backup and verify `close_rad = 0.55`.

3. **Settle the tilt curve**: single grasps at **25°** and **45°** (~2 min each). Smooth rise vs step change is the difference between "attenuation" and "threshold".

4. **Add the elongation < 1.5 guard** to `blob_axis.py`.

5. **Raise with Vincent on the 4th**: (a) the tilt finding and what it means for training data; (b) the Gaussian/normalisation mismatch (§8); (c) the sliding-window proposal (§4.2); (d) Newton lands end of August.

Short term

6. **Finish the kinematic test** (§7.3) — Berith asked for it.

7. **Verify the serpentine fix** with a repeat run (§6.2).

8. **Commanded-vs-actual placement check**. The repeatability test only catches *inconsistent* error; a systematic offset present in both runs is invisible to it. Compare `pad_actual_pos_m` against the commanded offsets in `gui_config_used.json`.

9. **Re-stitch the older runs** with current settings so sigma is comparable.

Then Block 3

10. **Produce one Vincent-compliant training pair:** centered, odd × odd, ≥ 22 mm travel per axis ($n_x = n_y = 2$, step 5.5 → 25 points, ~62 min, extension 11.0 mm every side = 2.08×2.00 taxels).

11. **Decide the pair format:** taxel resolution ($7 \times 4 \rightarrow 11 \times 8$) or the 0.75 mm canvas with the taxel grid as a crop boundary. *Changes Block 3's architecture — settle before generating volume.*

12. **Decide single vs temporal input.** Note the four temporal snapshots are currently near-useless: p05/p50/p95 land at frames **56, 58, 62** — 0.1 s apart. The close is far too fast relative to 60 Hz for them to be "progressive". Slowing the close is a **collector** change.

13. Scale up → train the U-Net → A/B against Paper 2.

12. WORKING PREFERENCES (for whoever picks this up)

- **One verifiable step at a time.** Measure, don't guess.
- **Complete files, not diffs.** Distances in mm.
- **Concise, direct answers** — one to three lines preferred.
- ****Kourosh's visual read of the simulation overrides diagnostics when they conflict. This has been correct every time.****
- Never present schematic pseudocode as a quotable line from a real file.
- Verify file freshness by grepping for a known recent signature before editing — uploads are not guaranteed to be current.
- **State predictions before running a test**, so the result can falsify them.

Several hypotheses this period were wrong and the data said so quickly: the hinge explanation for the row gain, baseline subtraction fixing that spread, the "strong lateral-position artifact" ($r = 0.94$ was a crude-metric artifact; PCA gives $r = 0.25\text{--}0.37$), and thicker-rod-shows-tilt-better.